

Laboratorio 2: Integración Continua con AWS CodeCommit y AWS CodeBuild utilizando Python

Introducción

La Integración Continua (Continuous Integration o CI) es una práctica ampliamente utilizada en DevOps que permite integrar cambios de código de manera frecuente y automática. Cada modificación realizada por un desarrollador puede ser compilada y validada mediante pruebas automatizadas, permitiendo detectar errores de forma temprana.

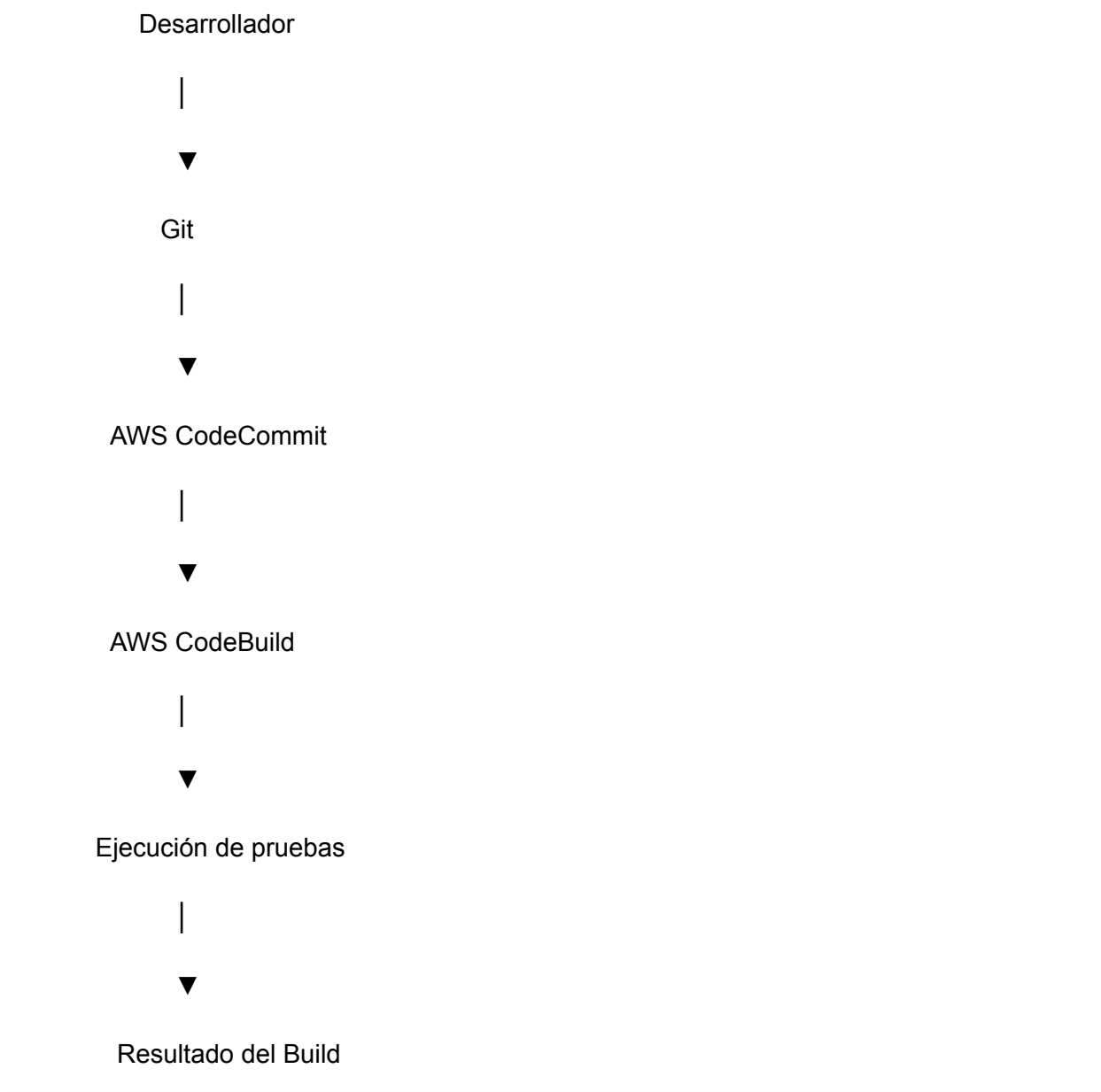
En este laboratorio se implementará un flujo básico de Integración Continua utilizando **AWS CodeCommit** como repositorio Git y **AWS CodeBuild** como servicio de compilación automática. Para simplificar el ejercicio se utilizará una aplicación desarrollada en Python, la cual podrá ejecutarse tanto en Windows como en macOS o Linux.

Objetivos

Al finalizar este laboratorio el estudiante será capaz de:

- Comprender el funcionamiento de AWS CodeCommit.
 - Crear un repositorio Git en AWS.
 - Crear una aplicación sencilla en Python.
 - Versionar el código mediante Git.
 - Configurar un proyecto en AWS CodeBuild.
 - Ejecutar pruebas automatizadas.
 - Analizar el resultado de una compilación.
 - Identificar errores durante el proceso de construcción.
-

Arquitectura del laboratorio



Requisitos

Antes de comenzar asegúrese de contar con:

- Una cuenta de AWS.
- Git instalado.
- Python 3 instalado.
- Visual Studio Code (opcional).
- AWS CLI (opcional).

Verifique la instalación ejecutando:

```
git --version  
python --version
```

Paso 1. Crear el proyecto

Crear una carpeta denominada

```
python-demo
```

Dentro de ella crear los siguientes archivos

```
python-demo  
|  
├── app.py  
├── test_app.py  
├── requirements.txt  
└── buildspec.yml
```

Paso 2. Crear la aplicación

Crear el archivo **app.py**

```
def saludar(nombre):  
    return (  
        f"Bienvenido al Diplomado de Ciberseguridad, {nombre}. "  
        "Esperamos que este laboratorio fortalezca tus conocimientos "  
        "sobre AWS CodeCommit y AWS CodeBuild."  
    )  
  
if __name__ == "__main__":  
    print(saludar("Estudiante"))
```

Guardar el archivo.

Paso 3. Crear la prueba

Crear el archivo

```
test_app.py
from app import saludar

def test_saludar():
    esperado = (
        "Bienvenido al Diplomado de Ciberseguridad, Roberto. "
        "Esperamos que este laboratorio fortalezca tus conocimientos "
        "sobre AWS CodeCommit y AWS CodeBuild."
    )

    assert saludar("Roberto") == esperado
```

Esta prueba verifica que la función entregue exactamente el mensaje esperado.

Paso 4. Crear las dependencias

Crear el archivo

requirements.txt

Agregar

pytest

Este archivo indica qué librerías debe instalar CodeBuild antes de ejecutar las pruebas.

Paso 5. Crear el archivo buildspec

Crear el archivo

buildspec.yml

Agregar el siguiente contenido

version: 0.2

phases:

install:

runtime-versions:

python: 3.11

commands:

- pip install -r requirements.txt

build:

commands:

- pytest

artifacts:

files:

- '**/*'

Este archivo le indica a AWS CodeBuild cómo ejecutar el proyecto.

Paso 6. Probar localmente

Abrir una terminal dentro del proyecto.

Instalar las dependencias.

```
pip install -r requirements.txt
```

Ejecutar

```
pytest
```

El resultado esperado será

```
=====
1 passed
=====
```

Esto confirma que el proyecto funciona correctamente antes de subirlo a AWS.

Paso 7. Creación del repositorio en AWS CodeCommit

En este paso se creará un repositorio Git administrado por AWS. Este repositorio almacenará el código fuente de la aplicación y será utilizado posteriormente por AWS CodeBuild como origen del proyecto.

7.1 Acceder a AWS CodeCommit

1. Inicie sesión en la consola de AWS.
2. En el cuadro **Search** ubicado en la parte superior de la consola escriba:

CodeCommit

3. Seleccione **AWS CodeCommit**.

La consola mostrará la página principal del servicio.

7.2 Crear un repositorio

Seleccione el botón

Create repository

Aparecerá la siguiente ventana de configuración.

Repository name

Ingrese

Laboratorio-Ciberseguridad

Este nombre identifica el repositorio dentro de la cuenta AWS.

Description

Ingrese

Repositorio para el laboratorio de Integración Continua.

La descripción es opcional, pero permite identificar el propósito del repositorio.

Tags

No es necesario agregar etiquetas para este laboratorio.

Finalmente seleccione

Create

Después de unos segundos AWS mostrará el mensaje

Repository successfully created

Ahora observará una pantalla similar a la siguiente:

Laboratorio-Ciberseguridad

Clone URL

HTTPS

SSH

Repository files

Branches

Pull Requests

Settings

En este momento el repositorio estará vacío.

7.3 Obtener la URL del repositorio

Seleccione

Clone URL

Elegir

Clone HTTPS

Copiar la dirección.

Por ejemplo

`https://git-codecommit.us-east-1.amazonaws.com/v1/repos/Laboratorio-Ciberseguridad`

Esta dirección será utilizada por Git para enviar el código fuente al repositorio.

Paso 8. Crear el proyecto en AWS CodeBuild

Una vez que el código haya sido enviado al repositorio, será posible configurar el servicio de compilación automática.

8.1 Abrir AWS CodeBuild

Desde la consola AWS buscar

CodeBuild

Seleccionar

AWS CodeBuild

En la página principal seleccionar

Create build project

8.2 Project Configuration

Project Name

Escriba

Laboratorio-CodeBuild

Este nombre identifica el proyecto de compilación.

8.3 Source

En esta sección se define desde dónde obtendrá el código fuente.

Source Provider

Seleccionar

AWS CodeCommit

Al hacerlo aparecerá una lista de repositorios.

Seleccionar

Laboratorio-Ciberseguridad

Branch

Seleccionar

main

CodeBuild descargará automáticamente el contenido de esta rama cada vez que se ejecute una compilación.

8.4 Environment

Esta sección define el sistema operativo y el entorno donde se ejecutará el Build.

Environment Image

Seleccionar

Managed Image

AWS administrará automáticamente la imagen utilizada para la compilación.

Operating System

Seleccionar

Amazon Linux

Runtime

Seleccionar

Standard

Image

Seleccionar la imagen más reciente disponible.

Runtime Version

Seleccionar

Python 3.11

Esta imagen ya incluye Python y Pip instalados.

8.5 Service Role

AWS necesita permisos para leer el repositorio y ejecutar la compilación.

Seleccionar

New Service Role

AWS creará automáticamente un rol IAM con los permisos necesarios.

No es necesario modificar ningún otro parámetro.

8.6 Buildspec

Seleccionar

Use buildspec.yml

CodeBuild utilizará el archivo almacenado en el repositorio para ejecutar las distintas fases de la compilación.

8.7 Artifacts

Seleccionar

No artifacts

En este laboratorio únicamente se ejecutarán pruebas automatizadas.

8.8 Logs

Mantener habilitado

Amazon CloudWatch Logs

Esto permitirá revisar el detalle de la ejecución una vez finalizado el Build.

8.9 Crear el proyecto

Revisar la configuración.

Finalmente seleccionar

Create Build Project

AWS mostrará el mensaje

Build project successfully created

8.10 Ejecutar el Build

Presionar

Start Build

Durante la ejecución se observarán las siguientes fases:

DOWNLOAD_SOURCE

INSTALL

PRE_BUILD

BUILD

POST_BUILD

UPLOAD_ARTIFACTS

FINALIZING

Si todas las pruebas son exitosas aparecerá el estado

SUCCEEDED

Resultados esperados

Al finalizar este laboratorio el estudiante habrá logrado:

- Crear un repositorio Git en AWS CodeCommit.
 - Administrar versiones utilizando Git.
 - Desarrollar una aplicación sencilla en Python.
 - Configurar AWS CodeBuild.
 - Ejecutar pruebas automatizadas mediante `pytest`.
 - Analizar los registros de una compilación.
 - Identificar errores durante el proceso de Integración Continua.
 - Comprender el flujo básico de CI utilizando servicios de AWS.
-

Conclusión

Durante este laboratorio se implementó un flujo básico de Integración Continua utilizando AWS CodeCommit y AWS CodeBuild. Se comprobó cómo un repositorio Git puede integrarse con un servicio de compilación administrado para validar automáticamente el código fuente antes de su despliegue.

Este tipo de automatización permite detectar errores de manera temprana, mejorar la calidad del software y reducir el tiempo necesario para integrar cambios en proyectos colaborativos. Estas herramientas forman parte del ecosistema DevOps de AWS y constituyen la base para implementar procesos más avanzados de Integración y Entrega Continua (CI/CD), incorporando posteriormente servicios como AWS CodePipeline, Amazon ECR, ECS o Elastic Beanstalk.